

project3/model.py

```

1  ## Building and training a bigram language model
2  from functools import partial
3  import math
4
5  import torch
6  import torch.nn as nn
7  from einops import einsum, reduce, rearrange
8
9  from config import BigramConfig, MiniGPTConfig
10
11 class BigramLanguageModel(nn.Module):
12     """
13     Class definition for a simple bigram language model.
14     """
15
16     def __init__(self, config):
17         """
18         Initialize the bigram language model with the given configuration.
19
20         Args:
21         config : BigramConfig (Defined in config.py)
22                 Configuration object containing the model parameters.
23
24         The model should have the following layers:
25         1. An embedding layer that maps tokens to embeddings. (self.embeddings)
26            You can use the Embedding layer from PyTorch.
27         2. A linear layer that maps embeddings to logits. (self.linear) **set bias
to True**
28         3. A dropout layer. (self.dropout)
29
30         NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
31         """
32
33         super().__init__()
34         # ===== TODO : START ===== #
35
36         self.embeddings = nn.Embedding(config.vocab_size, config.embed_dim)
37         self.linear = nn.Linear(config.embed_dim, config.vocab_size, bias=True)
38         self.dropout = nn.Dropout(config.dropout)
39
40         # ===== TODO : END ===== #
41
42         self.apply(self._init_weights)
43
44     def forward(self, x):
45         """
46         Forward pass of the bigram language model.
47
48         Args:
49         x : torch.Tensor
50             A tensor of shape (batch_size, 1) containing the input tokens.
51

```

```

52     Output:
53     torch.Tensor
54         A tensor of shape (batch_size, vocab_size) containing the logits.
55     """
56
57     # ===== TODO : START ===== #
58
59     emb = self.embeddings(x)
60     emb = self.dropout(emb)
61     logits = self.linear(emb)
62     return logits
63
64     # ===== TODO : END ===== #
65
66     def _init_weights(self, module):
67         """
68         Weight initialization for better convergence.
69
70         NOTE : You do not need to modify this function.
71         """
72
73         if isinstance(module, nn.Linear):
74             torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
75             if module.bias is not None:
76                 torch.nn.init.zeros_(module.bias)
77         elif isinstance(module, nn.Embedding):
78             torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
79
80     def generate(self, context, max_new_tokens=100):
81         """
82         Use the model to generate new tokens given a context.
83         We will perform multinomial sampling which is very similar to greedy
84         sampling,
85         but instead of taking the token with the highest probability, we sample
86         the next token from a multinomial distribution.
87
88         Remember in Bigram Language Model, we are only using the last token to
89         predict the next token.
90         You should sample the next token  $x_t$  from the distribution  $p(x_t | x_{t-1})$ .
91
92         Args:
93         context : torch.Tensor
94             The context is a sequence of tokens, where each token is an index in
95             the vocabulary.
96         max_new_tokens : int
97             The maximum number of new tokens to generate.
98
99         Output:
100        torch.Tensor
101            A tensor containing the generated tokens.
102        """
103
104        ### ===== TODO : START ===== ###

```

```

101
102     self.eval()
103     device = context.device
104     generated = context.tolist()
105     with torch.no_grad():
106         for _ in range(max_new_tokens):
107             last_token = torch.tensor([[generated[-1]]], device=device)
108             logits = self.forward(last_token)
109             logits = logits[0, -1, :]
110             probs = torch.softmax(logits, dim=-1)
111             next_token = torch.multinomial(probs, num_samples=1).item()
112             generated.append(next_token)
113     return torch.tensor(generated, device=device)
114
115     ### ===== TODO : END ===== ###
116
117
118 class SingleHeadAttention(nn.Module):
119     """
120     Class definition for Single Head Causal Self Attention Layer.
121
122     As in Attention is All You Need (https://arxiv.org/pdf/1706.03762)
123
124     """
125
126     def __init__(
127         self,
128         input_dim,
129         output_key_query_dim=None,
130         output_value_dim=None,
131         dropout=0.1,
132         max_len=512,
133     ):
134         """
135         Initialize the Single Head Attention Layer.
136
137         The model should have the following layers:
138         1. A linear layer for key. (self.key) **set bias to False**
139         2. A linear layer for query. (self.query) **set bias to False**
140         3. A linear layer for value. (self.value) # **set bias to False**
141         4. A dropout layer. (self.dropout)
142         5. A causal mask. (self.causal_mask) This should be registered as a
143         buffer.
144             - You can use the torch.tril function to create a lower triangular
145             matrix.
146             - In the skeleton we use register_buffer to register the causal mask as
147             a buffer.
148             This is typically used to register a buffer that should not to be
149             considered a model parameter.
150
151         NOTE : Please make sure that the causal mask is upper triangular and not
152         lower triangular (this helps in setting up the test cases, )
153         NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
154         """

```

```

150     super().__init__()
151
152     self.input_dim = input_dim
153     if output_key_query_dim:
154         self.output_key_query_dim = output_key_query_dim
155     else:
156         self.output_key_query_dim = input_dim
157
158     if output_value_dim:
159         self.output_value_dim = output_value_dim
160     else:
161         self.output_value_dim = input_dim
162
163     causal_mask = None # You have to implement this, currently just a
placeholder
164
165     # ===== TODO : START ===== #
166
167     self.key = nn.Linear(input_dim, self.output_key_query_dim, bias=False)
168     self.query = nn.Linear(input_dim, self.output_key_query_dim, bias=False)
169     self.value = nn.Linear(input_dim, self.output_value_dim, bias=False)
170     self.dropout = nn.Dropout(dropout)
171     causal_mask = torch.triu(
172         torch.ones(max_len, max_len, dtype=torch.bool), diagonal=1
173     )
174
175     # ===== TODO : END ===== #
176
177     self.register_buffer(
178         "causal_mask", causal_mask
179     ) # Registering as buffer to avoid backpropagation
180
181     def forward(self, x):
182         """
183         Forward pass of the Single Head Attention Layer.
184
185         Args:
186         x : torch.Tensor
187             A tensor of shape (batch_size, num_tokens, token_dim) containing the
input tokens.
188
189         Output:
190         torch.Tensor
191             A tensor of shape (batch_size, num_tokens, output_value_dim)
containing the output tokens.
192
193         Hint:
194         - You need to 'trim' the causal mask to the size of the input tensor.
195         """
196
197     # ===== TODO : START ===== #
198
199     B, T, _ = x.shape
200     k = self.key(x)

```

```

201         q = self.query(x)
202         v = self.value(x)
203
204         scores = q @ k.transpose(-2, -1) / math.sqrt(self.output_key_query_dim)
205         mask = self.causal_mask[:T, :T]
206         scores = scores.masked_fill(mask, float("-inf"))
207
208         attn = torch.softmax(scores, dim=-1)
209         attn = self.dropout(attn)
210         out = attn @ v
211         return out
212
213         # ===== TODO : END ===== #
214
215
216 class MultiHeadAttention(nn.Module):
217     """
218     Class definition for Multi Head Attention Layer.
219
220     As in Attention is All You Need (https://arxiv.org/pdf/1706.03762)
221     """
222
223     def __init__(self, input_dim, num_heads, dropout=0.1) → None:
224         """
225         Initialize the Multi Head Attention Layer.
226
227         The model should have the following layers:
228         1. Multiple SingleHeadAttention layers. (self.head_{i}) Use setattr to
229         dynamically set the layers.
230         2. A linear layer for output. (self.out) **set bias to True**
231         3. A dropout layer. (self.dropout) Apply dropout to the output of the out
232         layer.
233
234         NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
235         """
236         super().__init__()
237
238         self.input_dim = input_dim
239         self.num_heads = num_heads
240
241         # ===== TODO : START ===== #
242
243         head_dim = input_dim // num_heads
244         for i in range(num_heads):
245             setattr(
246                 self,
247                 f"head_{i}",
248                 SingleHeadAttention(
249                     input_dim,
250                     output_key_query_dim=head_dim,
251                     output_value_dim=head_dim,
252                     dropout=dropout,
253                 ),
254             )

```

```

253         self.out = nn.Linear(input_dim, input_dim, bias=True)
254         self.dropout = nn.Dropout(dropout)
255
256         # ===== TODO : END ===== #
257
258     def forward(self, x):
259         """
260         Forward pass of the Multi Head Attention Layer.
261
262         Args:
263         x : torch.Tensor
264             A tensor of shape (batch_size, num_tokens, token_dim) containing the
input tokens.
265
266         Output:
267         torch.Tensor
268             A tensor of shape (batch_size, num_tokens, token_dim) containing the
output tokens.
269         """
270
271         # ===== TODO : START ===== #
272
273         head_outputs = []
274         for i in range(self.num_heads):
275             head = getattr(self, f"head_{i}")
276             head_outputs.append(head(x))
277         concat = torch.cat(head_outputs, dim=-1)
278         out = self.out(concat)
279         out = self.dropout(out)
280         return out
281
282         # ===== TODO : END ===== #
283
284
285 class FeedForwardLayer(nn.Module):
286     """
287     Class definition for Feed Forward Layer.
288     """
289
290     def __init__(self, input_dim, feedforward_dim=None, dropout=0.1):
291         """
292         Initialize the Feed Forward Layer.
293
294         The model should have the following layers:
295         1. A linear layer for the feedforward network. (self.fc1) **set bias to
True**
296         2. A GELU activation function. (self.activation)
297         3. A linear layer for the feedforward network. (self.fc2) ** set bias to
True**
298         4. A dropout layer. (self.dropout)
299
300         NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
301         """
302         super().__init__()

```

```

303
304     if feedforward_dim is None:
305         feedforward_dim = input_dim * 4
306
307     # ===== TODO : START ===== #
308
309     self.fc1 = nn.Linear(input_dim, feedforward_dim, bias=True)
310     self.activation = nn.GELU()
311     self.fc2 = nn.Linear(feedforward_dim, input_dim, bias=True)
312     self.dropout = nn.Dropout(dropout)
313
314     # ===== TODO : END ===== #
315
316     def forward(self, x):
317         """
318         Forward pass of the Feed Forward Layer.
319
320         Args:
321         x : torch.Tensor
322             A tensor of shape (batch_size, num_tokens, token_dim) containing the
input tokens.
323
324         Output:
325         torch.Tensor
326             A tensor of shape (batch_size, num_tokens, token_dim) containing the
output tokens.
327         """
328
329         ### ===== TODO : START ===== ###
330
331         h = self.fc1(x)
332         h = self.activation(h)
333         h = self.fc2(h)
334         h = self.dropout(h)
335         return h
336
337         ### ===== TODO : END ===== ###
338
339
340     class LayerNorm(nn.Module):
341         """
342         LayerNorm module as in the paper https://arxiv.org/abs/1607.06450
343
344         Note : Variance computation is done with biased variance.
345
346         Hint :
347         - You can use torch.var and specify whether to use biased variance or not.
348         """
349
350         def __init__(self, normalized_shape, eps=1e-05, elementwise_affine=True) →
None:
351             super().__init__()
352
353             self.normalized_shape = (normalized_shape,)

```

```

354         self.eps = eps
355         self.elementwise_affine = elementwise_affine
356
357         if elementwise_affine:
358             self.gamma = nn.Parameter(torch.ones(tuple(self.normalized_shape)))
359             self.beta = nn.Parameter(torch.zeros(tuple(self.normalized_shape)))
360
361     def forward(self, input):
362         """
363         Forward pass of the LayerNorm Layer.
364
365         Args:
366         input : torch.Tensor
367             A tensor of shape (batch_size, num_tokens, token_dim) containing the
input tokens.
368
369         Output:
370         torch.Tensor
371             A tensor of shape (batch_size, num_tokens, token_dim) containing the
output tokens.
372         """
373
374         mean = None
375         var = None
376         # ===== TODO : START ===== #
377
378         mean = input.mean(dim=-1, keepdim=True)
379         var = input.var(dim=-1, keepdim=True, unbiased=False)
380
381         # ===== TODO : END ===== #
382
383         if self.elementwise_affine:
384             return (
385                 self.gamma * (input - mean) / torch.sqrt((var + self.eps)) +
self.beta
386             )
387         else:
388             return (input - mean) / torch.sqrt((var + self.eps))
389
390
391 class TransformerLayer(nn.Module):
392     """
393     Class definition for a single transformer layer.
394     """
395
396     def __init__(self, input_dim, num_heads, feedforward_dim=None):
397         super().__init__()
398         """
399         Initialize the Transformer Layer.
400         We will use prenorm layer where we normalize the input before applying the
attention and feedforward layers.
401
402         The model should have the following layers:
403         1. A LayerNorm layer. (self.norm1)

```



```

404     2. A MultiHeadAttention layer. (self.attention)
405     3. A LayerNorm layer. (self.norm2)
406     4. A FeedForwardLayer layer. (self.feedforward)
407
408     NOTE : PLEASE KEEP OF EACH LAYER AS PROVIDED BELOW TO FACILITATE TESTING.
409     """
410
411     # ===== TODO : START ===== #
412
413     self.norm1 = LayerNorm(input_dim)
414     self.attention = MultiHeadAttention(input_dim, num_heads)
415     self.norm2 = LayerNorm(input_dim)
416     self.feedforward = FeedForwardLayer(input_dim, feedforward_dim)
417
418     # ===== TODO : END ===== #
419
420     def forward(self, x):
421         """
422         Forward pass of the Transformer Layer.
423
424         Args:
425         x : torch.Tensor
426             A tensor of shape (batch_size, num_tokens, token_dim) containing the
input tokens.
427
428         Output:
429         torch.Tensor
430             A tensor of shape (batch_size, num_tokens, token_dim) containing the
output tokens.
431         """
432
433         # ===== TODO : START ===== #
434
435         x = x + self.attention(self.norm1(x))
436         x = x + self.feedforward(self.norm2(x))
437         return x
438
439         # ===== TODO : END ===== #
440
441
442     class MiniGPT(nn.Module):
443         """
444         Putting it all together: GPT model
445         """
446
447         def __init__(self, config) → None:
448             super().__init__()
449             """
450             Putting it all together: our own GPT model!
451
452             Initialize the MiniGPT model.
453
454             The model should have the following layers:

```

```

455     1. An embedding layer that maps tokens to embeddings.
      (self.vocab_embedding)
456     2. A positional embedding layer. (self.positional_embedding) We will use
      learnt positional embeddings.
457     3. A dropout layer for embeddings. (self.embed_dropout)
458     4. Multiple TransformerLayer layers. (self.transformer_layers)
459     5. A LayerNorm layer before the final layer. (self.prehead_norm)
460     6. Final language Modelling head layer. (self.head) We will use weight
      tying (https://paperswithcode.com/method/weight-tying) and set the weights of the
      head layer to be the same as the vocab_embedding layer.

```

```

461
462     NOTE: You do not need to modify anything here.
463     """

```

```

464
465     self.config = config
466     self.vocab_embedding = nn.Embedding(config.vocab_size, config.embed_dim)
467     self.positional_embedding = nn.Embedding(
468         config.context_length, config.embed_dim
469     )
470     self.embed_dropout = nn.Dropout(config.embed_dropout)
471
472     self.transformer_layers = nn.ModuleList(
473         [
474             TransformerLayer(
475                 config.embed_dim, config.num_heads, config.feedforward_size
476             )
477             for _ in range(config.num_layers)
478         ]
479     )
480
481     # prehead layer norm
482     self.prehead_norm = LayerNorm(config.embed_dim)
483
484     self.head = nn.Linear(
485         config.embed_dim, config.vocab_size
486     ) # Language modelling head
487
488     if config.weight_tie:
489         self.head.weight = self.vocab_embedding.weight
490
491     # precreate positional indices for the positional embedding
492     pos = torch.arange(0, config.context_length, dtype=torch.long)
493     self.register_buffer("pos", pos, persistent=False)
494
495     # Needed by _init_weights for GPT-2 style FFN init scaling
496     self.num_layers = config.num_layers
497
498     self.apply(self._init_weights)

```

```

500     def forward(self, x):

```

```

501         """

```

```

502         Forward pass of the MiniGPT model.

```

```

503
504         Remember to add the positional embeddings to your input token!!

```

```

505
506     Args:
507     x : torch.Tensor
508         A tensor of shape (batch_size, seq_len) containing the input tokens.
509
510     Output:
511     torch.Tensor
512         A tensor of shape (batch_size, seq_len, vocab_size) containing the
logits.
513
514     Hint:
515     - You may need to 'trim' the positional embedding to match the input
sequence length
516     """
517
518     ### ===== TODO : START ===== ###
519
520     B, T = x.shape
521     tok_emb = self.vocab_embedding(x)
522     pos_emb = self.positional_embedding(self.pos[:T])
523     h = tok_emb + pos_emb
524     h = self.embed_dropout(h)
525     for layer in self.transformer_layers:
526         h = layer(h)
527     h = self.prehead_norm(h)
528     logits = self.head(h)
529     return logits
530
531     ### ===== TODO : END ===== ###
532
533 def _init_weights(self, module):
534     """
535     Weight initialization for better convergence.
536
537     NOTE : You do not need to modify this function.
538     """
539
540     if isinstance(module, nn.Linear):
541         if module._get_name() == "fc2":
542             # GPT-2 style FFN init
543             torch.nn.init.normal_(
544 self.num_layers)
545         )
546     else:
547         torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
548         if module.bias is not None:
549             torch.nn.init.zeros_(module.bias)
550     elif isinstance(module, nn.Embedding):
551         torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
552
553 def generate(self, context, max_new_tokens=100):
554     """
555     Use the model to generate new tokens given a context.

```

```

556
557     Args:
558     context : torch.Tensor
559         The context is a sequence of tokens, where each token is an index in
the vocabulary.
560     max_new_tokens : int
561         The maximum number of new tokens to generate.
562
563     Output:
564     torch.Tensor
565         A tensor containing the generated tokens.
566
567     Hint:
568     - This should be similar to the Bigram Language Model, but you will use
the entire context to predict the next token.
569         Instead of sampling from the distribution  $p(x_t | x_{t-1})$ ,
570         you will sample from the distribution  $p(x_t | x_{t-1}, x_{t-2}, \dots, x_{t-n})$ ,
571         where  $n$  is the context length.
572     - When decoding for the next token, you should use the logits of the last
token in the input sequence.
573
574     """
575
576     ### ===== TODO : START ===== ###
577
578     self.eval()
579     device = context.device
580     if context.dim() == 1:
581         generated = context.unsqueeze(0)
582     else:
583         generated = context
584     with torch.no_grad():
585         for _ in range(max_new_tokens):
586             x_cond = generated[:, -self.config.context_length:]
587             logits = self.forward(x_cond)
588             logits = logits[:, -1, :]
589             probs = torch.softmax(logits, dim=-1)
590             next_token = torch.multinomial(probs, num_samples=1)
591             generated = torch.cat([generated, next_token], dim=1)
592     return generated
593
594     ### ===== TODO : END ===== ###
595
596     def generate_beam(self, context, max_new_tokens=100, beam_width=5,
length_penalty=1.0):
597         """
598         Beam search decoder (Bonus).
599
600         Args:
601             context : 1D tensor (T,) or 2D (1, T) of token ids.
602             max_new_tokens : how many tokens to generate.
603             beam_width : number of beams  $k$ .
604             length_penalty : alpha used in score /  $\text{length}^\alpha$ .

```

```
605
606     Returns:
607         best_seq : 1D tensor of the highest-scoring sequence.
608     """
609     self.eval()
610     device = context.device
611     if context.dim() == 1:
612         context = context.unsqueeze(0)
613
614     beams = [(context, 0.0)]
615
616     with torch.no_grad():
617         for _ in range(max_new_tokens):
618             candidates = []
619             for seq, score in beams:
620                 x_cond = seq[:, -self.config.context_length:]
621                 logits = self.forward(x_cond)
622                 logits = logits[:, -1, :]
623                 log_probs = torch.log_softmax(logits, dim=-1)
624
625                 topk_logp, topk_idx = log_probs.topk(beam_width, dim=-1)
626                 for b in range(beam_width):
627                     next_tok = topk_idx[0, b].view(1, 1)
628                     next_logp = topk_logp[0, b].item()
629                     new_seq = torch.cat([seq, next_tok], dim=1)
630                     new_score = score + next_logp
631                     candidates.append((new_seq, new_score))
632
633             candidates.sort(
634                 key=lambda x: x[1] / ((x[0].shape[1]) ** length_penalty),
635                 reverse=True,
636             )
637             beams = candidates[:beam_width]
638
639     beams.sort(
640         key=lambda x: x[1] / ((x[0].shape[1]) ** length_penalty),
641         reverse=True,
642     )
643     best_seq, _ = beams[0]
644     return best_seq.squeeze(0)
645
```